

Databases & **Web Programming**

Week 8: Introduction to REST, JHipster &
SpringBoot, JDL

Overview: web programming

- Week 7: Introduction to Web Programming
- **Week 8: Backend: REST, SpringBoot and JDL**
- Week 9: Backend: SpringBoot constraints
- Week 10: class test
- Week 11: Front end: HTML, Angular

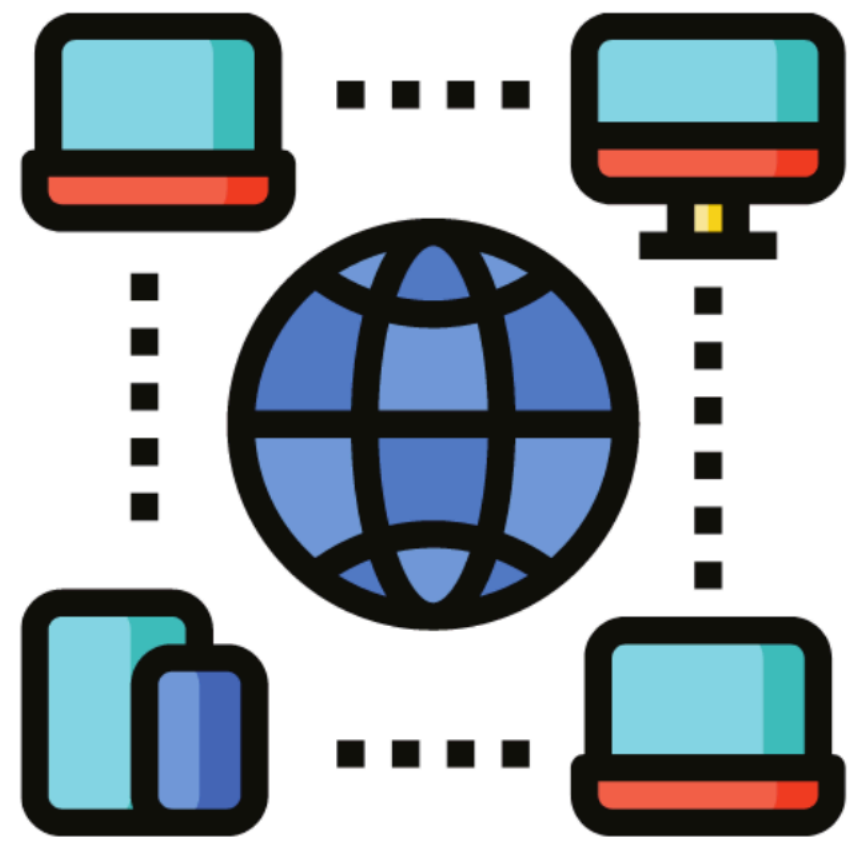
Reminders

- Why Web Programming
- Essential Protocols
- Terminal Commands
- Web application Architectures



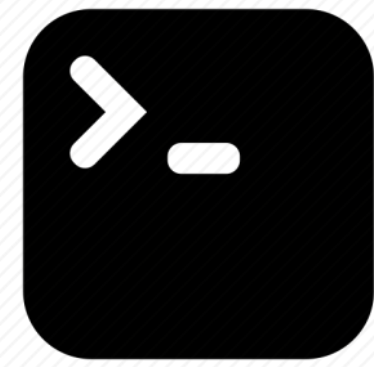
Summary: Why web programming?

- **Web:** HTTP+HTML accessed via browsers
- **Web pages:** Static
- **Web applications:** Dynamic
- **Careers and Employability:** full stack developer relevant to many roles in placement



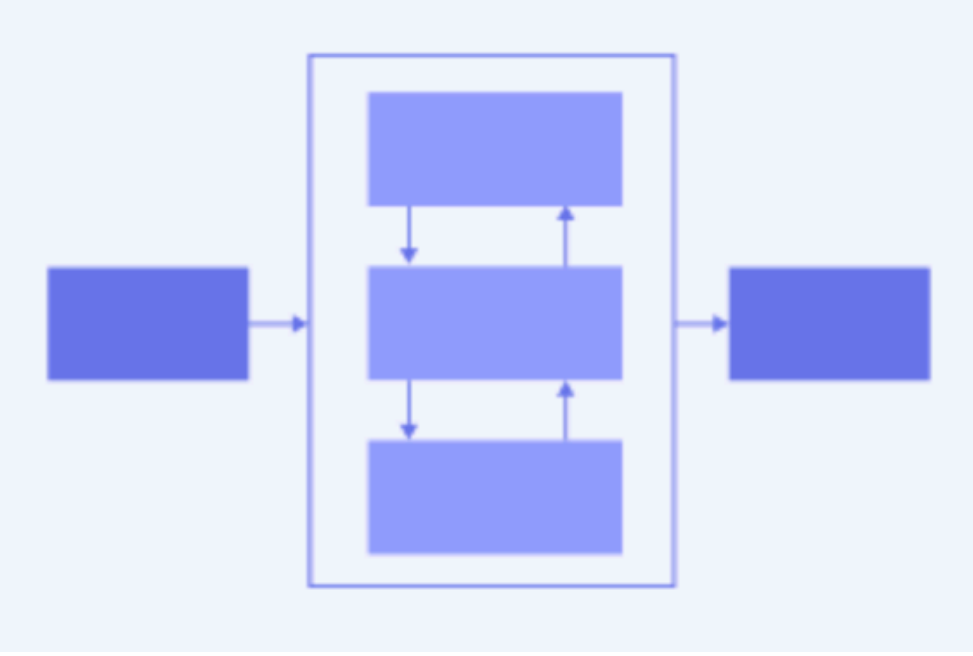
Summary: essential protocols

- **TCP/IP** - reliable packets sending / over internet
- **DNS** - "phone book" to find IPs from domain names
- **HTTP/S** - text based, stateless, web protocol, with methods
- **HTML/CSS/JS** - text based files to represent web front ends



Summary: terminal apps for the Internet

- The Terminal: command line interface apps
- **ifconfig**: configure/view your IP address
- **nslookup**: DNS lookup
- **ping**: check to a server is reponding
- **traceroute**: trace the path of routes
- **curl**: download from a url
- **telnet**: use text based protocols
- **ssh**: for secure, remote terminal access



Summary: Web application Architectures

- **Browsers and Web servers** - client-server architecture
- **CGI-bin** - dynamic apps, not scaleable
- **MVC** - integrated frontend & backend
- **REST** - scalable, modern
- **Tech stacks** - ∞ , SpringBoot/Java, Postgres/SQL, Angular/TS

This week

- REST API - Architecture Pattern
- JHipster & SpringBoot essentials
- JDL for SpringBoot generation



REpresentational State Transfer Application Programming Interfaces

- What is and an API, REST
- HTTP Methods vs CRUD operations
- REST Design Conventions
- REST Error Codes & Documentation

What are **APIs**

Contract of what the program/library/system **will provide**

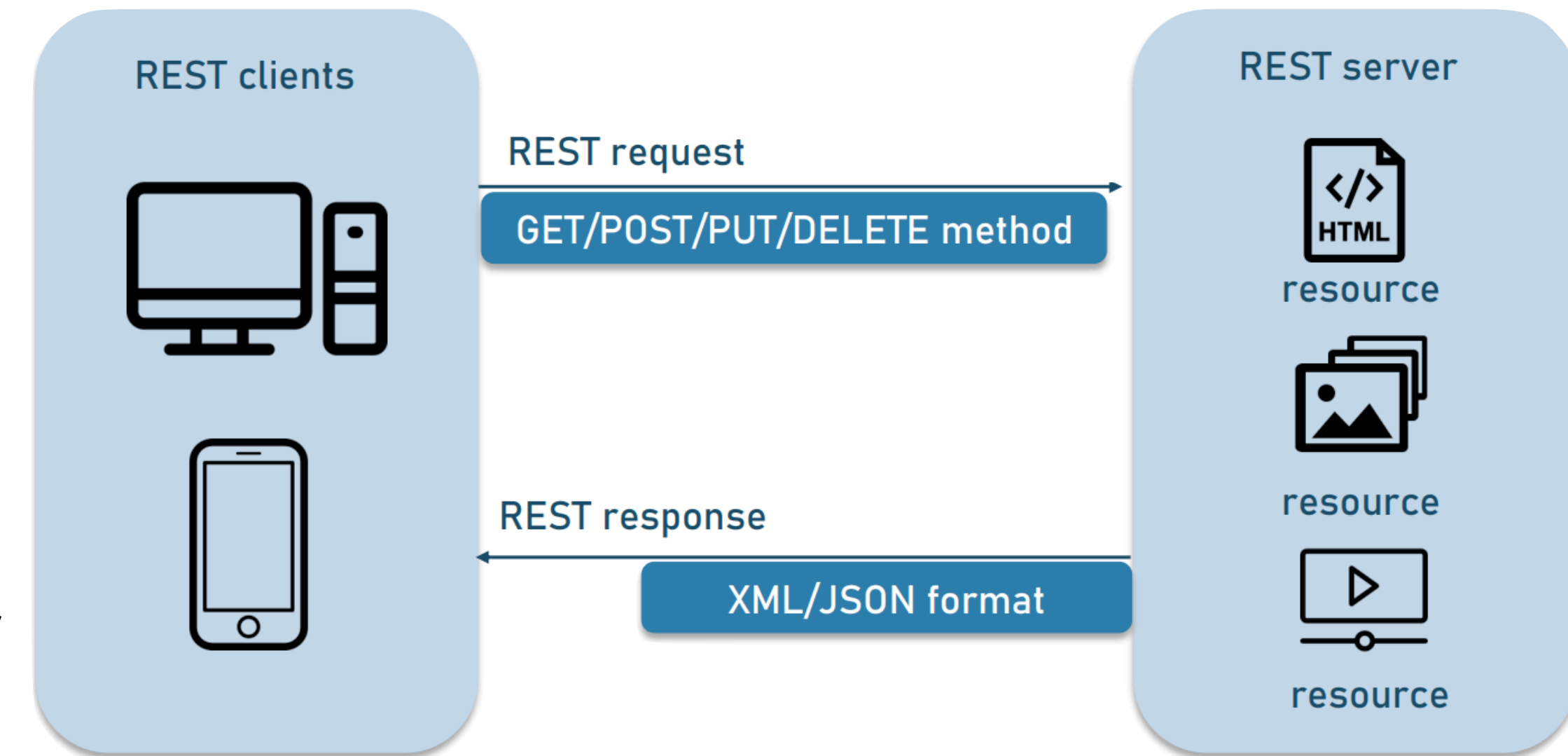
- API or Application Programming Interface:
 - Java API - Java Development Kit (JDK)
 - Standard library classes, interfaces, methods, and fields
 - Java Collections API (ArrayList<>); I/O (System.out.println)
 - REST: Architectural style for web applications
 - We design the API for our web applications

Analogy:   a chef preps recipe (program), the waiter takes order  (API), someone consumes the food (data) 

Question: in the analogy, who owns the restaurant?



What are **RESTful APIs**



- **REpresentational State Transfer**
- API or Application Programming Interface with:
 - a base URI, such as `http://example.com/api`
 - standard HTTP methods (e.g., GET, POST, PUT, and DELETE);
 - a media type that defines data elements (JSON/XML)
- REST API+Documentation: skeleton generated by JHipster from JDL



Roy Fielding

HTTP 1.0 and REST standards

Fielding's RESTful PhD Thesis - six constraint for RESTful:

- **Uniform Interface - design conventions**
- Stateless
- Cacheable
- Client-server
- Layered System
- (optional) Code on demand



https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

HTTP protocol methods & CRUD

HTTP method	CRUD equivalent	Description
GET	Read	Get a representation of the state of the requested resource.
PUT	Update (replace)	Overwrite an existing resource - the target is replaced by the representation enclosed in the request.
POST	Create	Create a new target resource with the state of the resource enclosed in the request.
DELETE	Delete	Delete the target resource.
PATCH	Update (modify)	Update an existing resource - the server processes the patch representation enclosed in the request.

HTTP REST headers with curl -v

```
curl -v https://restcountries.com/v3.1/name/brazil
```

Request header:

```
> GET /v3.1/name/brazil HTTP/2
> Host: restcountries.com
> User-Agent: curl/8.9.1
> Accept: */*
```

Response header & body:

```
< HTTP/2 200
< server: nginx/1.22.1
< date: Sun, 16 Nov 2025 20:30:21
GMT
< content-type: application/json
< content-length: 3789
< cache-control: public, immutable,
max-age=31556926
<
[
  {
    "name": {
      "common": "Brazil",
      "official": "Federative
```


REST API Design Conventions

For endpoints

Use nouns (entities)

do not include verbs (actions)

Task	Method	Incorrect ❌	Correct ✅
List all todo notes	GET	/GetAllTodos	/todo
Create a todo	POST	/createTodo	/todo
Delete a todo note	DELETE	/deleteTodo	/todo/145
Get a todo's subject	GET	/getTodoSubject	/todo/10/subject

REST API Design Conventions:

plural not singular

- REST rarely used for single unique resources.
- Keep it simple: use only plural for resources:

`/todos` **instead of** `/todo`

`/users` **instead of** `/user`

`/items` **instead of** `/item`

`/vehicles` **instead of** `/vehicle`

REST API Design Conventions: GET is stateless

Use PUT, POST and DELETE methods to change state

GET should never change the state of an item :

GET /item/923/publish ❌

GET /item/923?publish ❌ don't use query parameters for REST operations

POST /item/923/publish ✅

REST API Design Conventions: formats

Specify the format (JSON, XML, HTML) in the headers

Content-Type: defines the request format

Accept: defines a list of acceptable response formats

REST API design conventions: object relations

Use sub-resources to get related resources

GET `/todo/923/item`

(a list of all items for todo #923)

GET `/todo/923/item/4`

(from todo #923, get item #4 only)

REST API design conventions:

versioning

Publishing an API requires careful thought about app structure, but what if our domain changes

- Make the API version mandatory, use vN
 - so `/api/v1/` 
- Use a whole number
 - so no `/api/v1.1/` 
- Have different APIs for different apps or microservices
- so `/users/api/v3/`  and `/shop/api/v4/` 

REST API design conventions: sort

- Allow ascending and descending sort using parameters(?) + -
 - GET `/items?-cost,+color`

Returns a list of items sorted descending by cost and ascending by color

REST API design conventions: paging

- Allow clients to flexibly deal with large results sets
 - `GET /items?limit=20,offset=10`

Returns 20 items, does not include the the first 10 results

By default : no limit, offset 0

REST API design conventions: aliasing

move useful/commonly used tasks into more easily accessed paths

Instead of:

- `POST /users/create/` ❌

Consider:

- `POST /users/` ✅

REST: HTTP status codes

REST APIs use the **Status-Line** part of an HTTP response message to inform clients of their request's overarching result.

five categories of HTTP status codes for the result of a request:

- 1xx: Informational – Protocol-level information.
- 2xx: Success – The request was accepted successfully.
- 3xx: Redirection – Some additional action specified in the code for the client to complete the request.
- 4xx: Client Error – These error status codes point to a client issue.
- 5xx: Server Error – The server takes responsibility the error status.

<https://restfulapi.net/http-status-codes/>

REST: most common request codes

200 OK

201 Created

204 No Content

304 Not Modified

400 Bad Request

403 Forbidden

409 Conflict

401 Unauthorized

404 Not Found

500 Internal Server Error

<https://www.restapitutorial.com/httpstatuscodes>

REST API Documentation Tools

- Swagger UI + OAS Editor - OAS 3.0 to HTML, JavaScript, and CSS, example (provided by JHipster)
- ReDoc - OAS 2.0/3.0, rich UI, browser or Docker based, example
- DapperDox - OAS 2.0/3.0., enrich with markdown, detailed user docs, used by uk.gov
- OpenAPI Generator - OAS 2.0/3.0, server stub generator, support/docs, HTML/Cwiki, example



Summary: REST APIs

- APIs are contracts, REST for web app APIs
- 9 REST design conventions
- Common REST Error Codes
- Common REST Documentation tools



🦴🦴🦴 Backend Skeleton & Generator 🦴🦴🦴🦴

- The What, Why and Who of JHipster/SpringBoot
- BoilerPlate Code
- Books and Documentation
- Overview of the generated CRUD app template



JHipster is a tech-stack generator

- Backend: Springboot
- Frontends: Angular or React or Vue
- Databases: PostgreSQL or MySQL, noSQL

Remember: re-generate the template app to change settings e.g. frontend, database, JDL etc

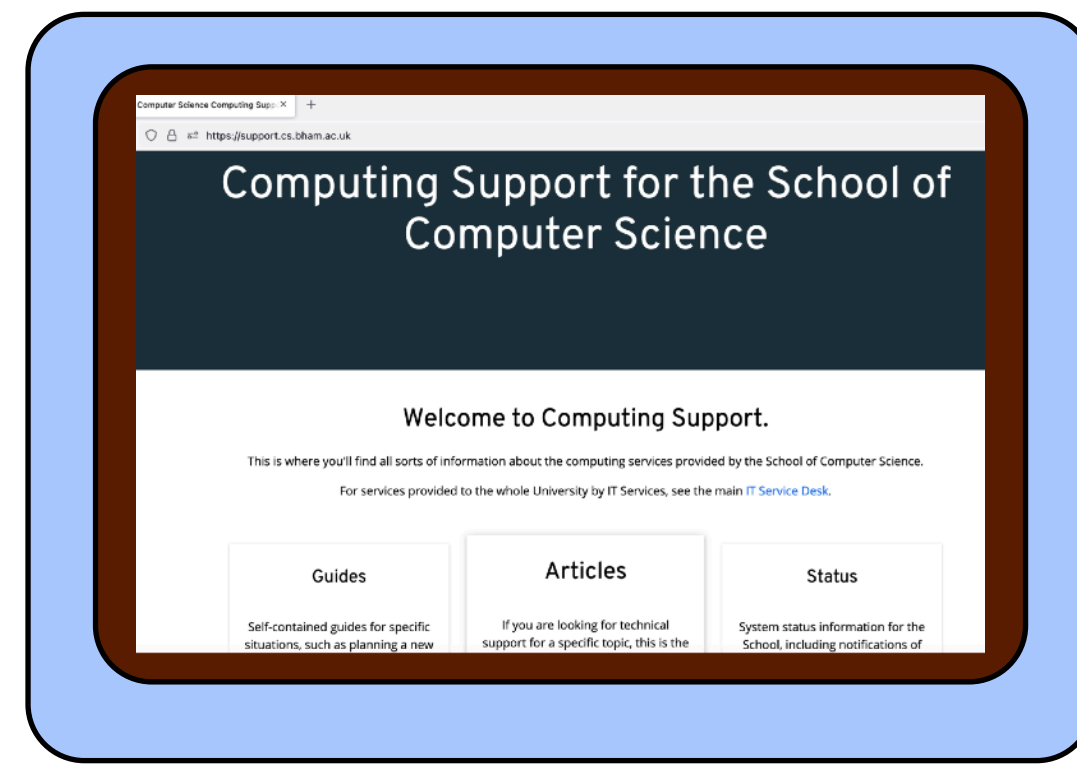


- Options when generating an application
 `.yo-rc.json`

```
{  
  "generator-jhipster": {  
    "applicationType": "monolith",  
    "authenticationType": "jwt",  
    "baseName": "friday",  
    "buildTool": "maven",  
    "cacheProvider": "ehcache",  
    "clientFramework": "angular",  
    "clientTestFrameworks": [],  
    "clientTheme": "darkly",  
    "clientThemeVariant": "dark",  
    "creationTimestamp": 1763128010809,  
    "databaseType": "sql",  
    "devDatabaseType": "h2Disk",  
  }  
}
```

jwtSecretKey: must be **private** and **unique** to avoid
Token Forgery (impersonating users), Elevation of Privileges, Session hijacking

Browser

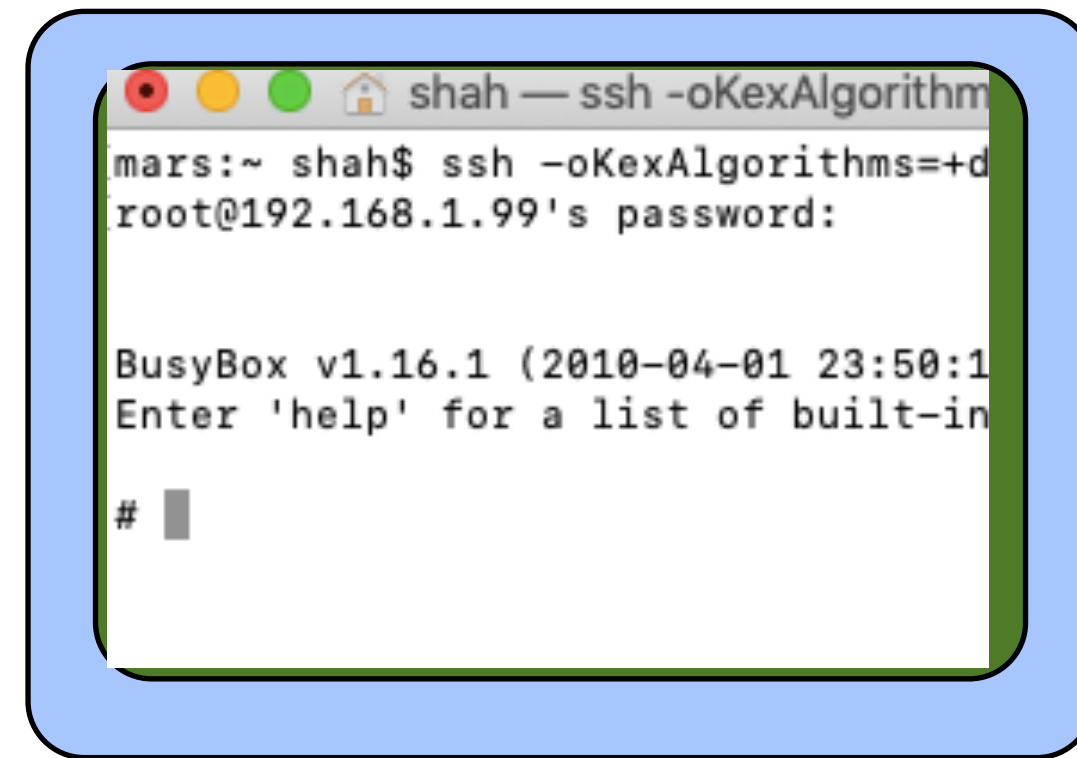


Front End
e.g. Angular



↓ ↑ HTTP/REST

Web
Server



Back End
e.g. SpringBoot/Java

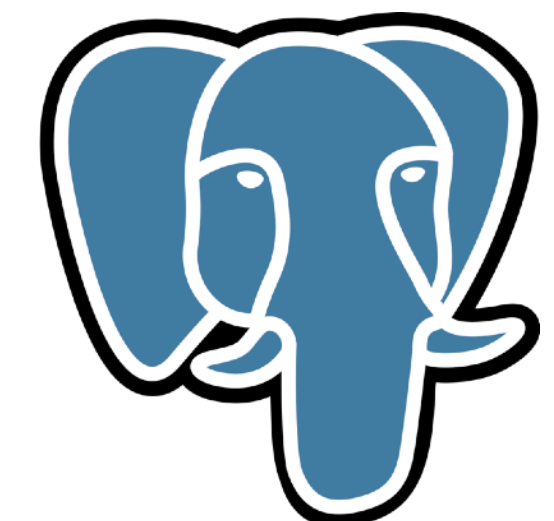


↓ ↑ SQL

Database
Server



Database
e.g. PostgreSQL DB



JHipster Resources

Documentation

<https://www.jhipster.tech/getting-started>

JDL docs

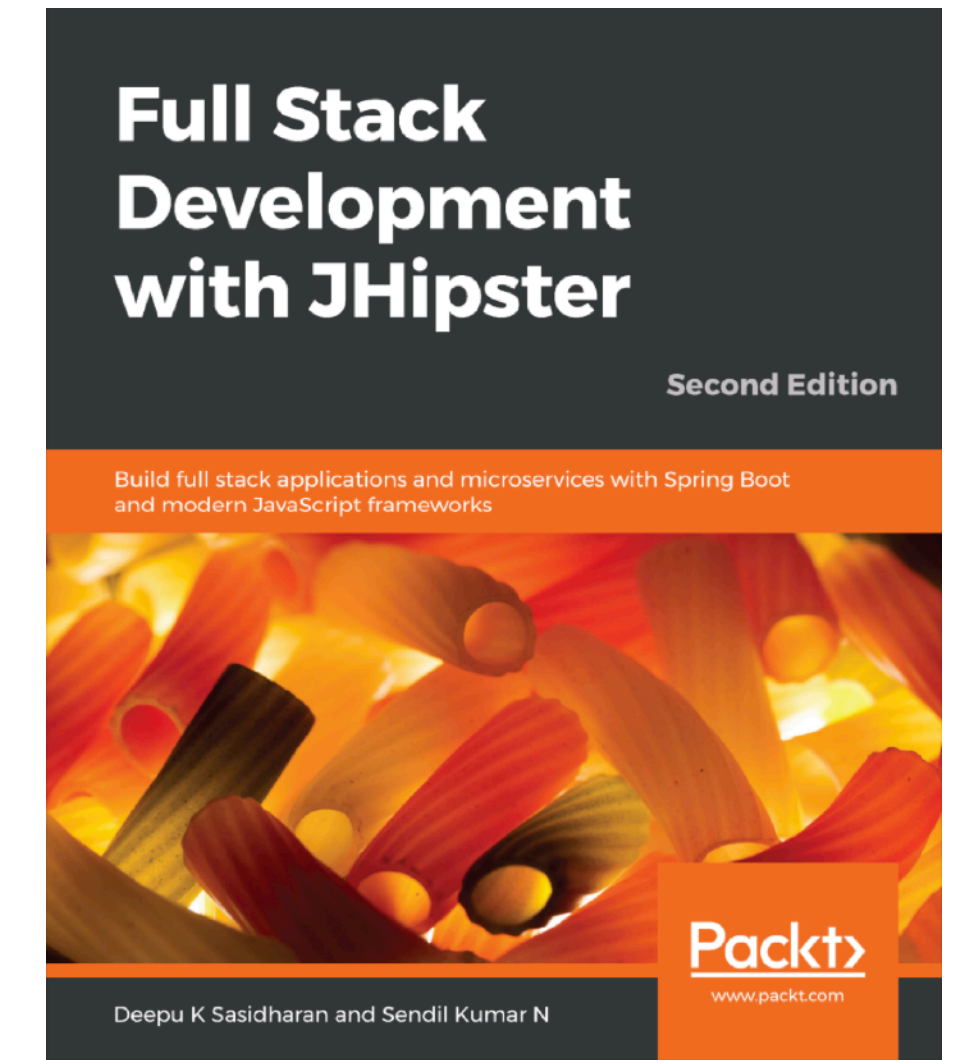
<https://www.jhipster.tech/jdl/getting-started>

JDL samples

<https://github.com/jhipster/jdl-samples>

- The JHipster 7 Mini Book
- Full stack development with JHipster 6

canvas.bham.ac.uk/courses/83763/pages/jhipster-books



What is JHipster



- Generates templates of **modern, production-ready apps**
- **Boilerplate:** JDL to Angular, SpringBoot and Postgresql (etc)
- Starting point for development
- Includes admin interface, supports micro services, monoliths
- Created by Julien Dubois - Java Dev Relations at Microsoft

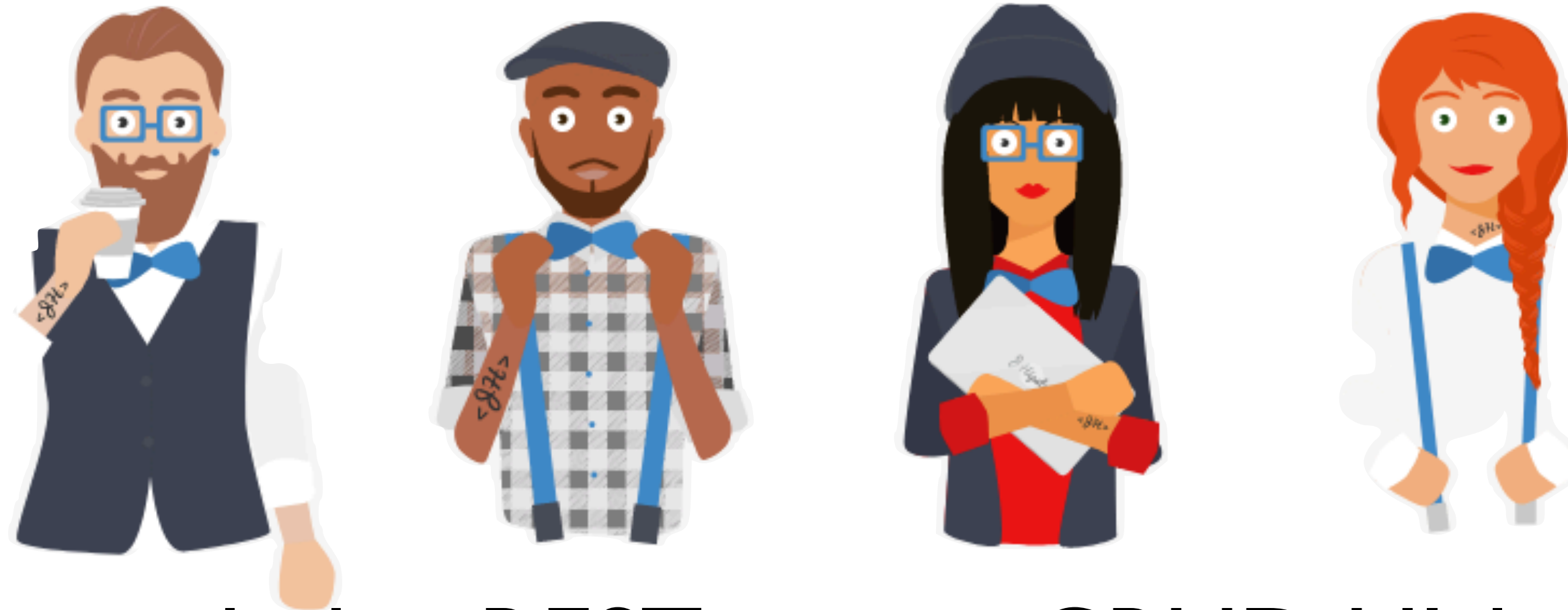


Boilerplate Code in Java, Spring Boot & JHipster

- **Boilerplate:** repetitive code, adds no business value, required by language (e.g. Java).
- **Spring Boot:** Simplifies boilerplate via auto-conf, embedded servers, and pre-built components for quicker development.
- **JHipster:** Reduces boilerplate even more by generating entire Spring Boot-based applications with pre-configured settings, databases, and UI.

Analogy: Compare boilerplate to rental contracts - what changes each time?

Why JHipster



- Save 100+ hours. includes, REST server, CRUD UI, login and admin UI
 - develop useful, agile apps quickly
- Follow industry standards
 - Supports other languages, frameworks (node, .net, micronaut, qarkus etc)
- **Simulate real world jobs: rare to set your own stack**
 - **Excellent template to learn best practice**

Using JHipster JDL:

quickly generate a template CRUD app

- **C**reate
- **R**ead
- **U**pdate
- **D**elete

Posts

This covers 70-80% of what users use and developers code

Refresh list

+ Create a new Po

A new Post is created with identifier 1052

ID ^	Title ⇅	Content ⇅	Date ⇅	Blog ⇅	
1051	Hello World	Hi!	28 Apr 2021 00:00:00	Admin's blog	View Edit Delete
1052	JHipster is awesome!	I just love it!	28 Apr 2021 08:00:00	User's blog	View Edit Delete

Who uses



And around 65,000 others including: JPMorgan Chase & Co., Ford, Fidelity, Oracle, Deloitte, Capgemini, Cognizant, Tata Group, Accenture

What is



Backend Framework: for Java-based server-side scalable dynamic web apps.



Dynamic Data: Integrates with databases to deliver real-time, personal content via **RESTful APIs**.

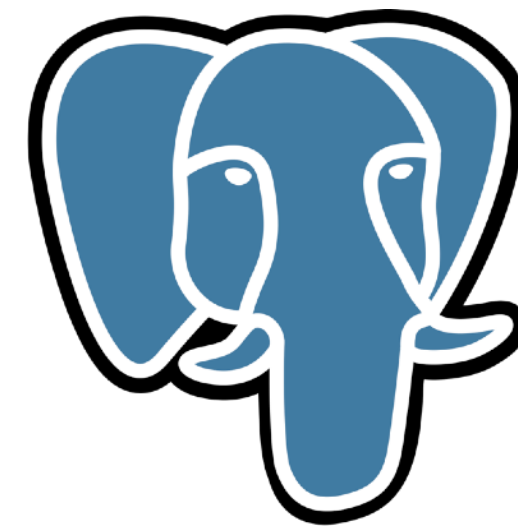


Rapid Development as it Provides:

- dependency injection
- embedded servers (e.g., Tomcat)
- auto-configuration,
- pre-built components for faster development.

Supported Databases

- **PostgreSQL - production**



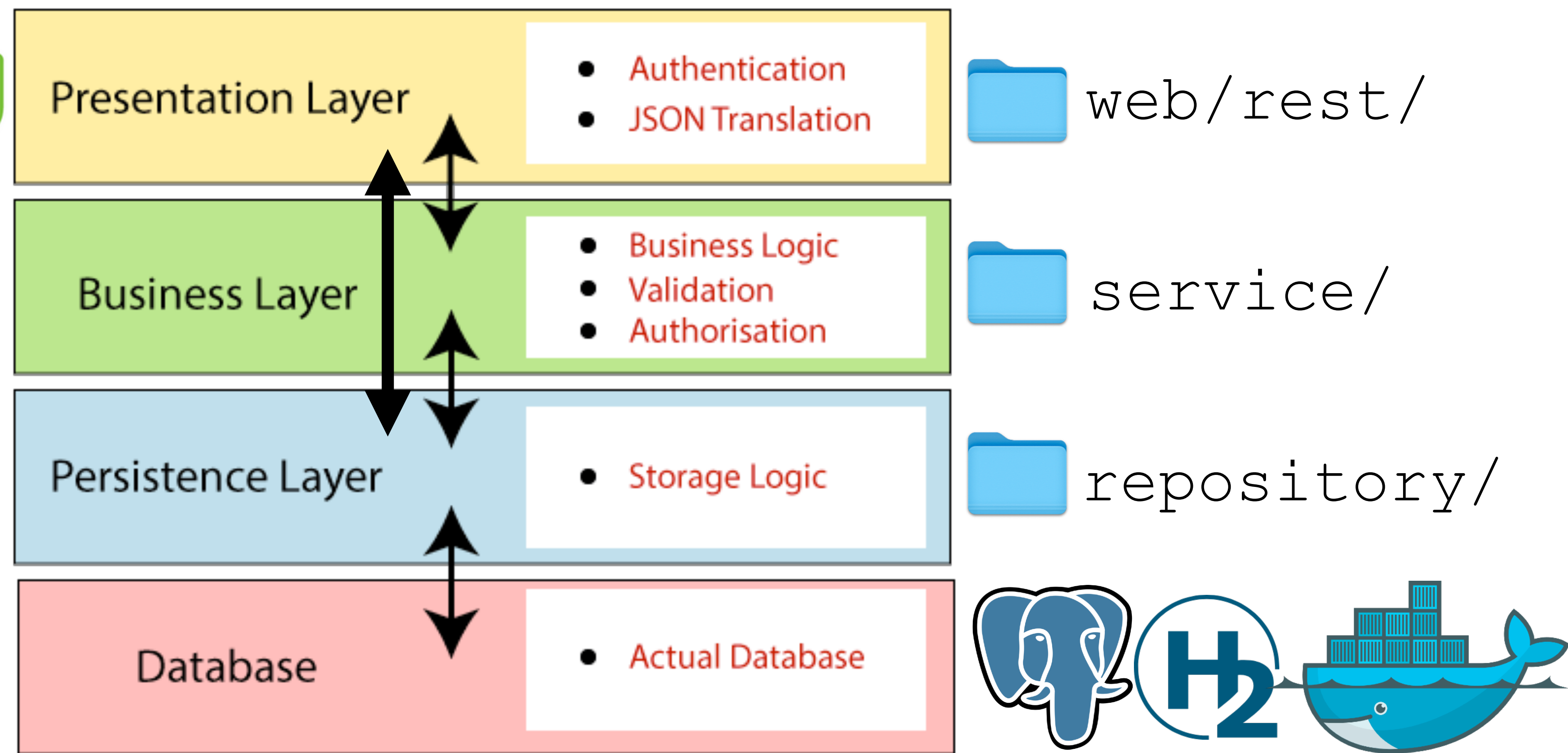
- MySQL/MariaDB



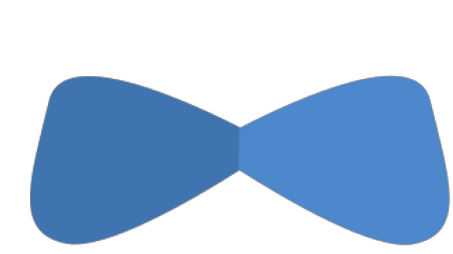
- Oracle

- **H2 - dev mode**





- Spring Boot a framework for building dynamic web apps
 - Boot is "opinionated" decisions made or "booted-up" for us
 - JHipster generates a project template
- Java framework: lots of libraries and tool support



Hipster Backend Tech

- Java/Spring Boot: classes, program logic, respond to requests via **REST**
- Hibernate/JPA: **Object Relational Mapping** SQL rows to Java objects
- Liquibase: **database changesets** to create/evolve the database schema
- FakerJS: random **dummy data** to populate our application
- Swagger: **interface and documenting** our REST API
- Authentication and Authorisation: JWT, OAuth, tokens for REST
- CI/CD: for continuous integration and deployment to a **cloud/VM**
- Docker: to package our app to **easily run anywhere** and deploy

WE WILL START CODING THE TEMPLATE NEXT WEEK

Summary & JHipster

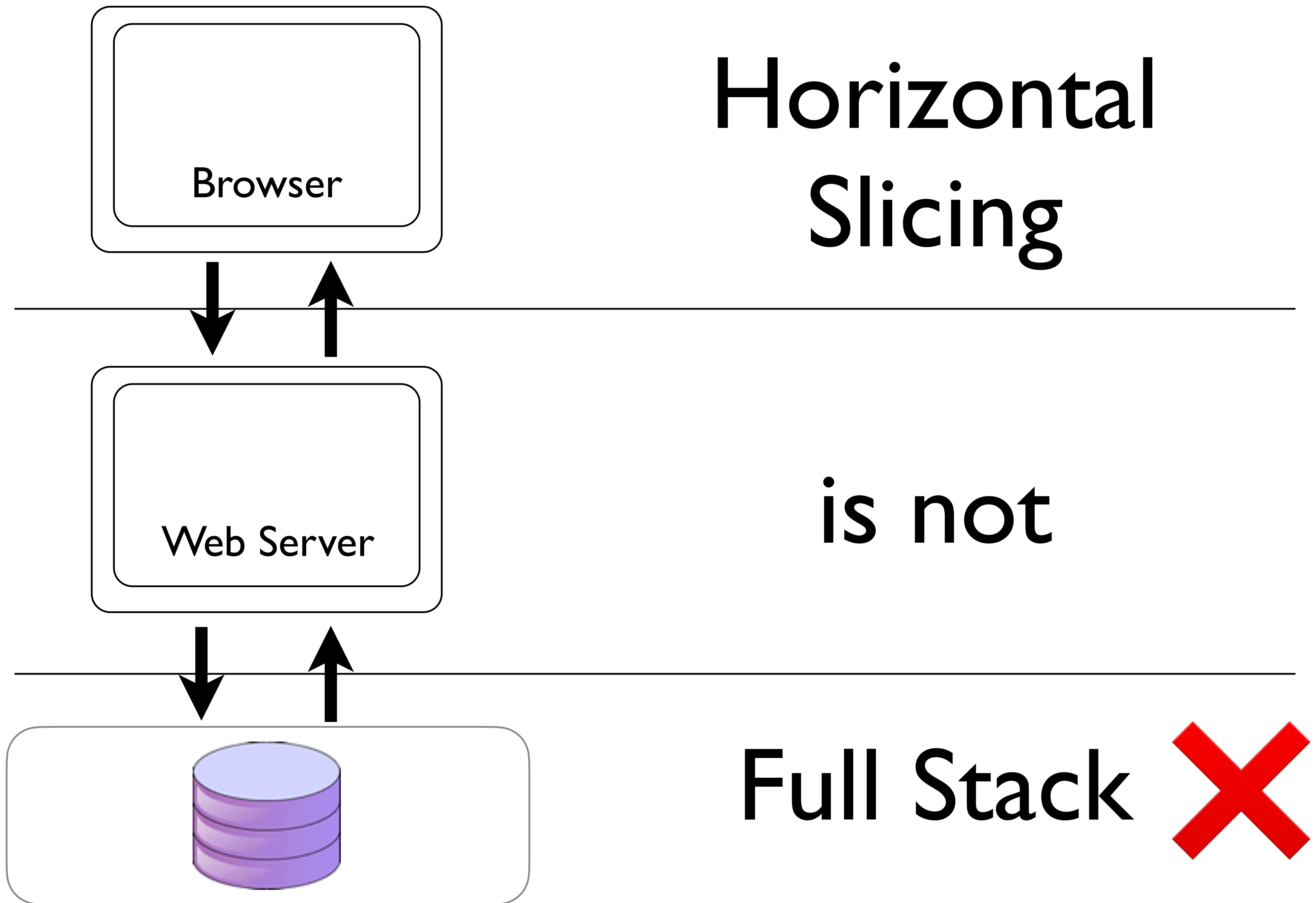
- JHipster: generate modern app templates, avoid coding boilerplate, save time
- Structure of a Generated template: SpringBoot+Tech list
- CRUD app: what JHipster gives us from JDL



- Reminder: JDL Entities and Relationships
- Vertical vs Horizontal Slicing
- Entities vs Features
- JDL User, commands and options

JHipster Domain Language (JDL)

- For entities (like Customer, Product), fields+types (price, name) and relationships between entities.
- Human-readable, visual and separate from the code
- **Generates app template:** JHipster auto generates Spring Boot classes, REST, database migration scripts, UI.
- **Also has:** app config, scroll, pagination or service layers, dto.
- **The JDL file serves as a central, documented blueprint for the entire data model.**

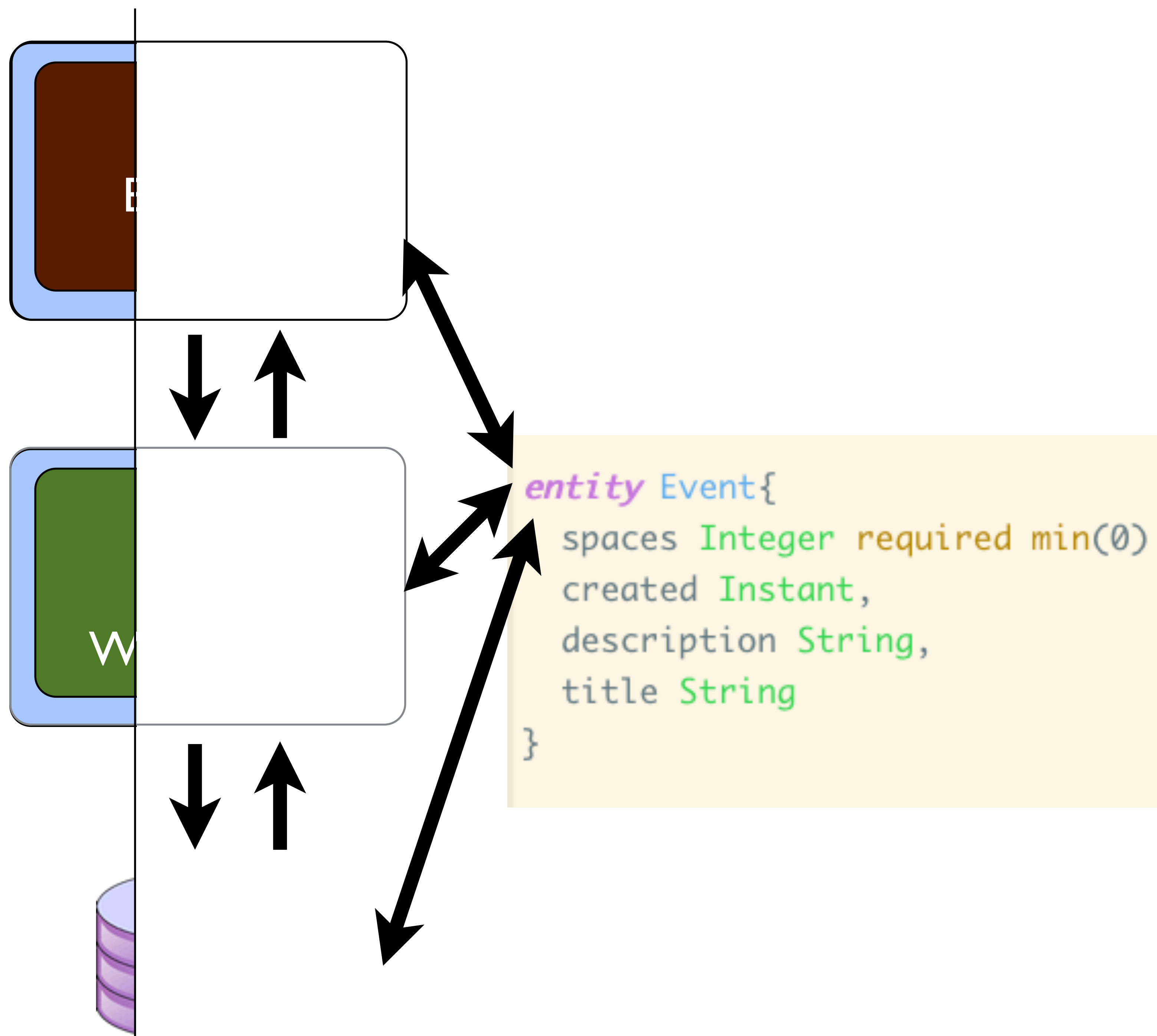


Vertical Slicing

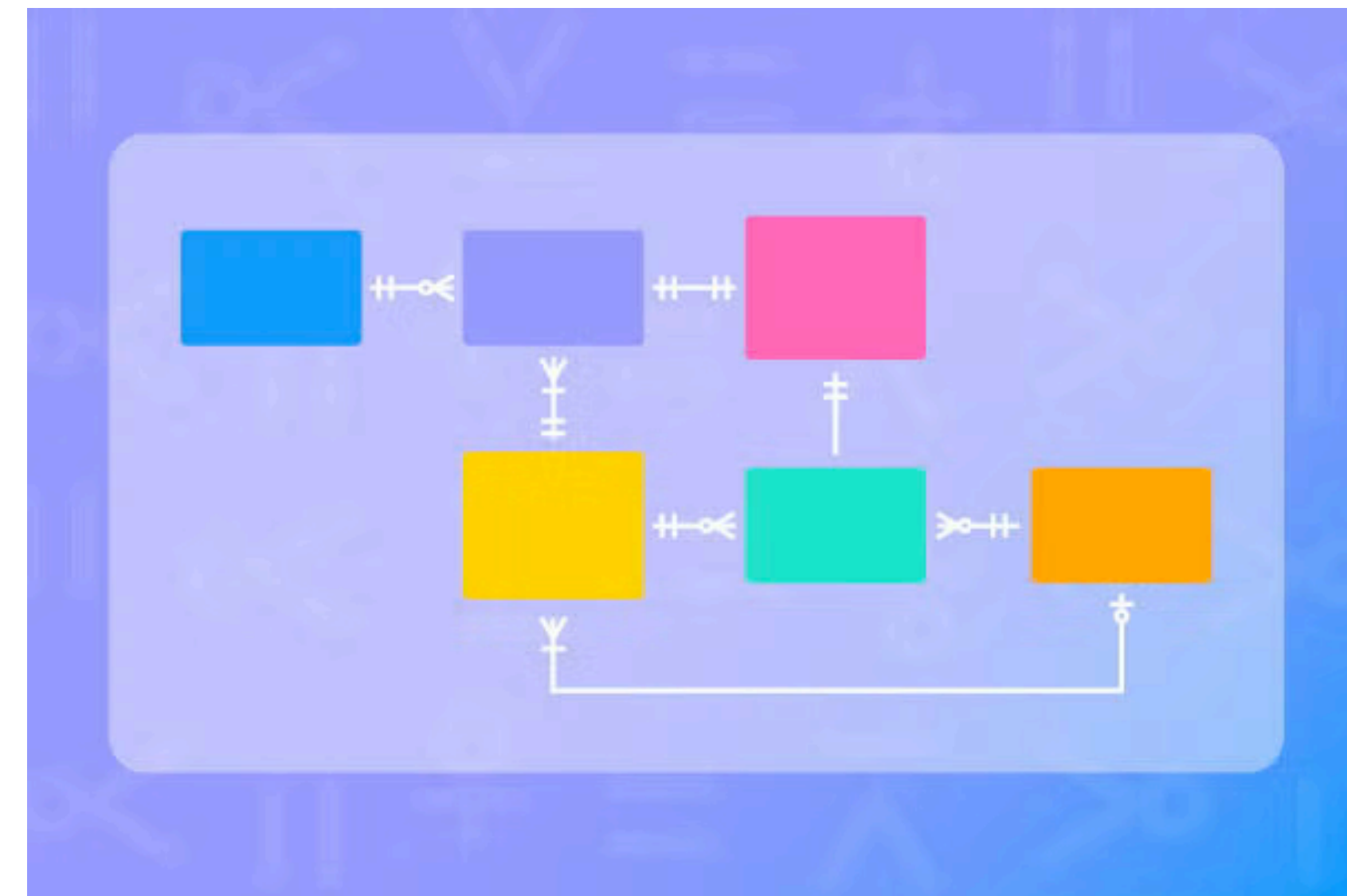


via

Entities



What is an Entity



- **In the Problem Domain:** An entity is a *type* or *domain concept* or *business object*
- **In the Database:**
 - A **JDL entity** maps directly to a SQL **table**:
 - Each **JDL field** becomes a **column** in that table
 - Each **row** in that table is **an object of that type**
- **Backend:** SpringBoot, database read/write, and REST API
- **Frontend and CRUD:** Angular components - views to create/edit/delete items

Example Entities for an App

"Our app will be used by a society to organise events"

Developer	Entity	Description
Ali	User Profile	Has pics, bio and uni verification
Bobby	Society Profile	Has details about the organisers
Charlie	Events	Planned event, location, capacity
Danny	Badge	Users collect badges of attended events
Eddie	Comment	Comments on a Event, Society or User
Franky	Check-In	At an Event take a register of attendees
Gem	Photo Gallery	Event, Society, User have a Gallery
Harry	Review	Users can rate Events they attended

JDL Entity Examples

Book	Customer	Order	Product	Event
title : String* author : String* publishedDate : Instant price : BigDecimal description : TextBlob coverImage : ImageBlob	firstName : String* lastName : String* email : String* phoneNumber : String dateOfBirth : Instant loyaltyPoints : Integer profilePicture : ImageBlob	orderNumber : String* orderDate : Instant* totalAmount : BigDecimal* shippingAddress : TextBlob customerId : Long	name : String* category : String* price : BigDecimal* stockQuantity : Integer* description : TextBlob productImage : ImageBlob	eventName : String* eventDate : Instant* location : String* ticketPrice : BigDecimal eventDescription : TextBlob eventPoster : ImageBlob

```
entity Book {  
  title String required,  
  author String required,  
  publishedDate Instant,  
  price BigDecimal,  
  description TextBlob,  
  coverImage ImageBlob  
}
```

```
entity Customer {  
  firstName String required,  
  lastName String required,  
  email String required,  
  phoneNumber String,  
  dateOfBirth Instant,  
  loyaltyPoints Integer,  
  profilePicture ImageBlob  
}
```

```
entity Order {  
  orderNumber String required,  
  orderDate Instant required,  
  totalAmount BigDecimal required,  
  shippingAddress TextBlob,  
  customerId Long  
}
```

```
entity Product {  
  name String required,  
  category String required,  
  price BigDecimal required,  
  stockQuantity Integer required,  
  description TextBlob,  
  productImage ImageBlob  
}
```

```
entity Event {  
  eventName String required,  
  eventDate Instant required,  
  location String required,  
  ticketPrice BigDecimal,  
  eventDescription TextBlob,  
  eventPoster ImageBlob  
}
```


JDL relations: Library app

```
entity Author {  
  name String  
}
```

```
entity Book {  
  title String  
}
```

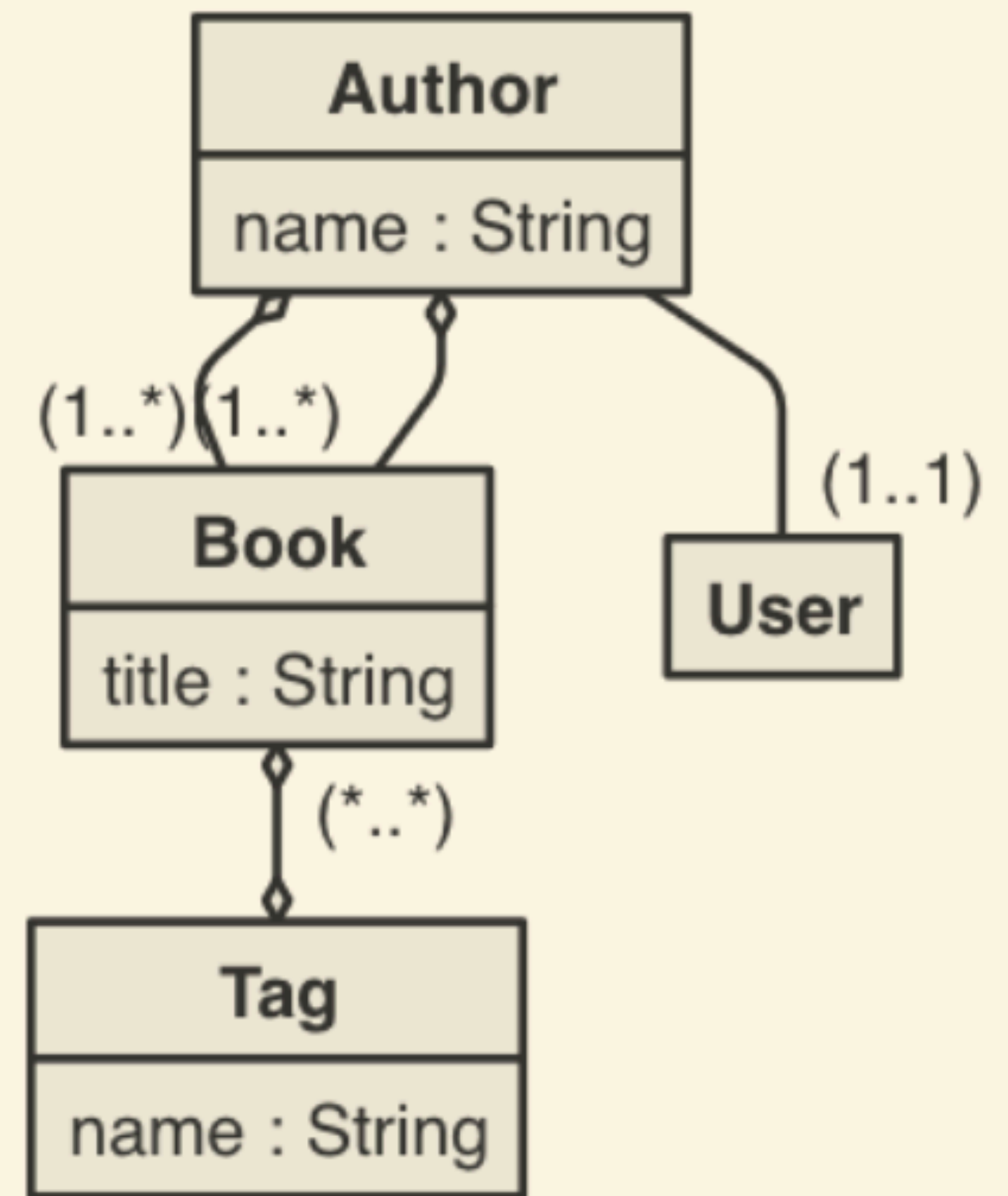
```
entity Tag {  
  name String  
}
```

```
relationship OneToOne {  
  Author{user} to User with builtInEntity  
}
```

```
relationship OneToMany {  
  Author{books} to Book  
}
```

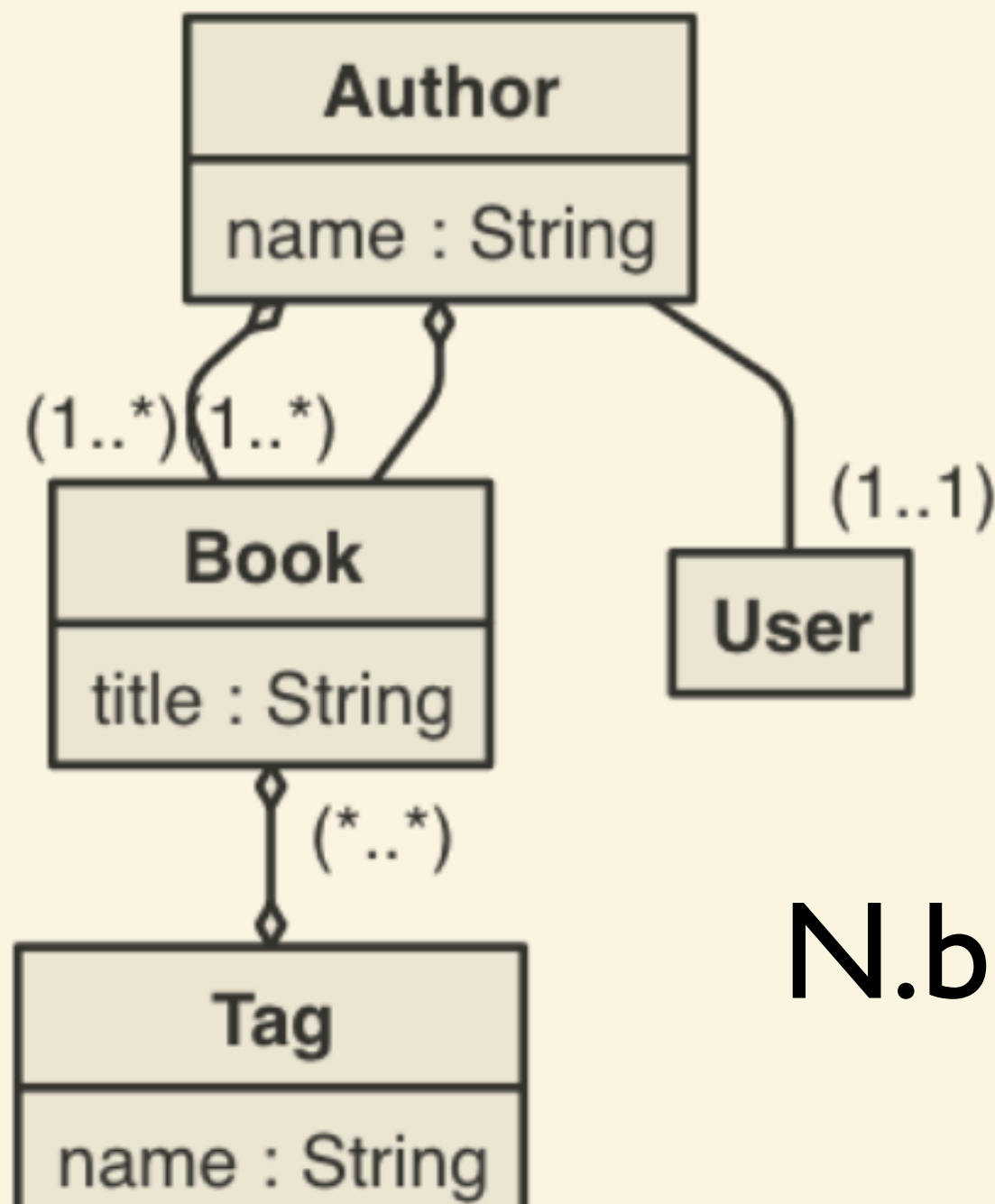
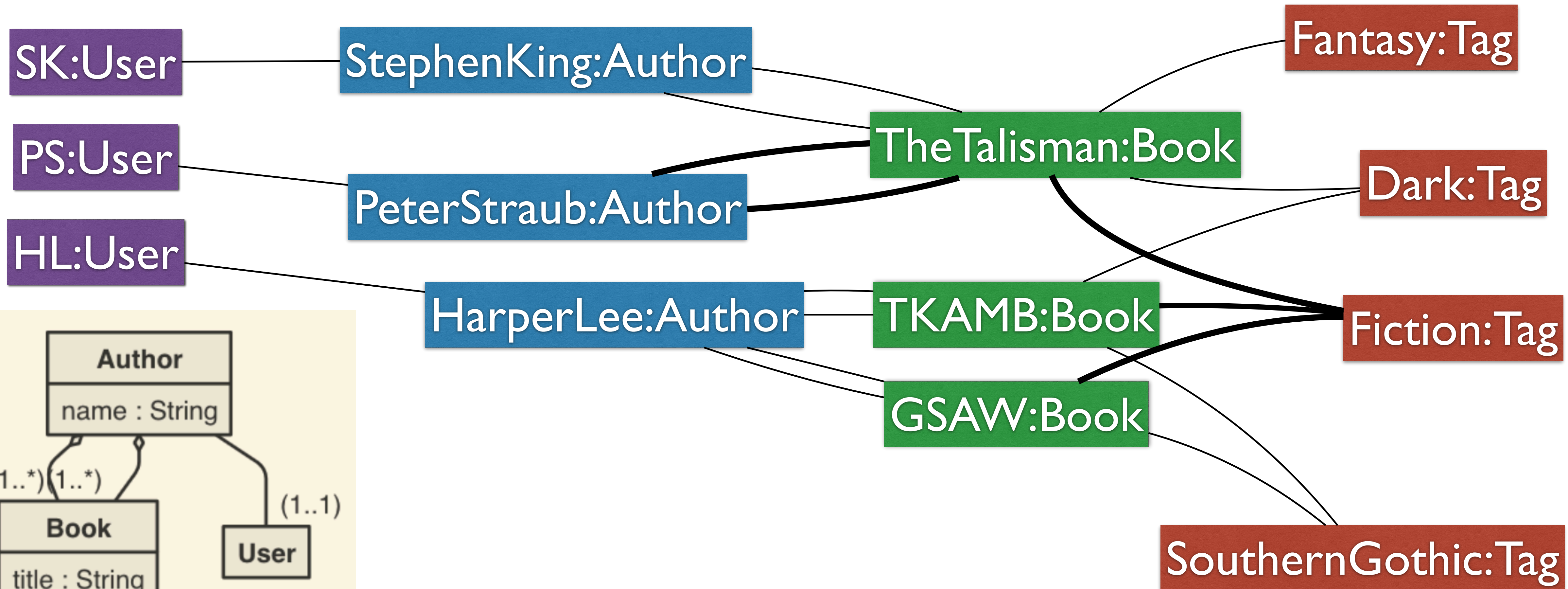
```
relationship ManyToOne {  
  Book{author} to Author  
}
```

```
relationship ManyToMany {  
  Book{tags} to Tag  
}
```



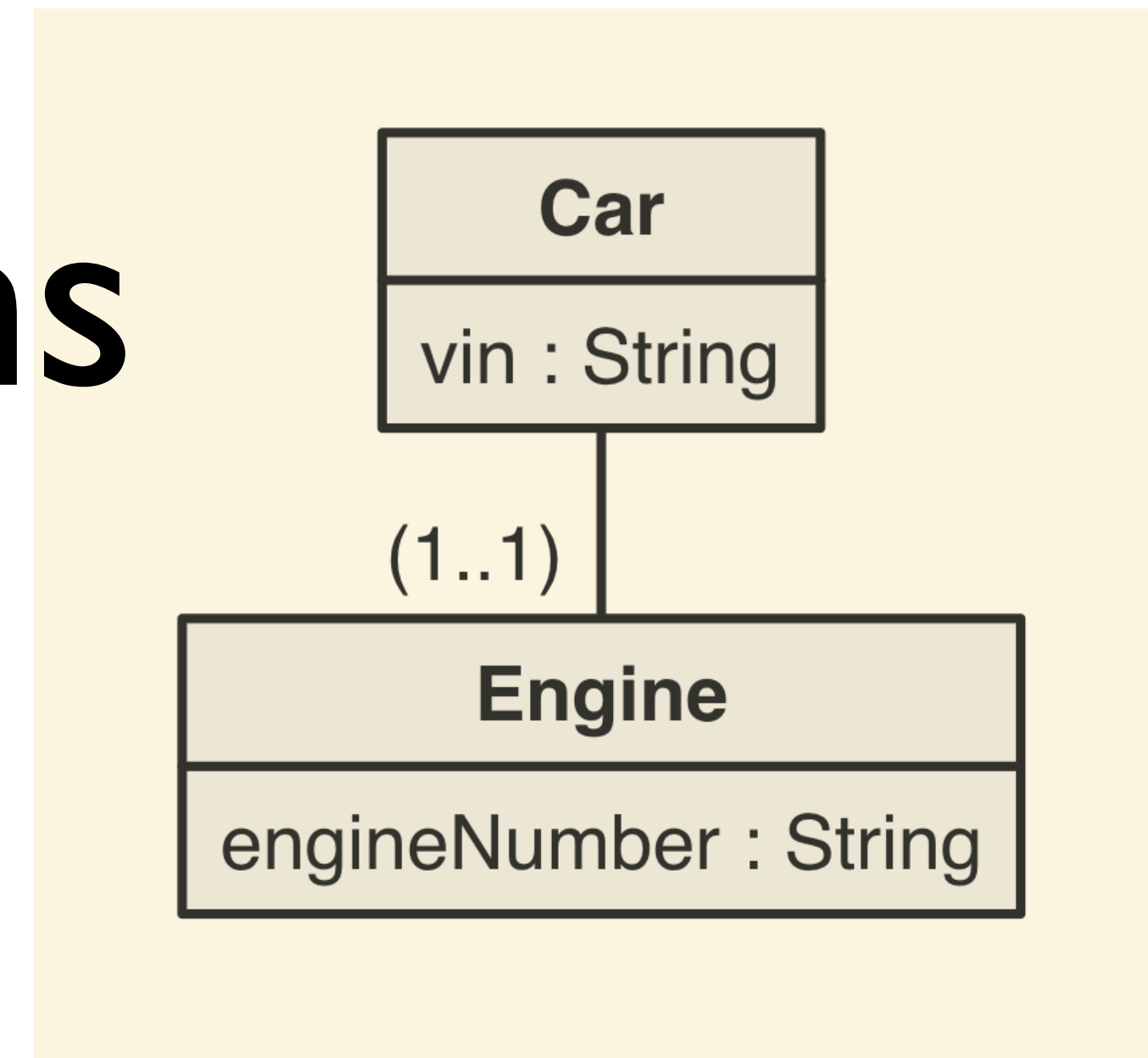
N.b. User is built in- User links our apps to JHipster login/authorities

JDL relationships: example



N.b. JHipster does not fully maintain bi-directional relationships

Customise Relations



```
entity Car{
  vin String
}
entity Engine{
  engineNumber String
}
```

```
relationship OneToOne {
  Car{theEngine(engineNumber)} to Engine
}
```

Form Label:

customised to “The Engine”

Instance identity:

customised to “Engine Number” from Engine

The screenshot shows a form for the Car entity. It has a title bar with the text "Car". Below the title bar, there are two input fields: "ID" with the value "1" and "Vin" with the value "VVWWZZZ1234XX5678". Below these fields, there is a label "The Engine" followed by a blue link "KK1234". At the bottom of the form, there are two buttons: a purple button with a left arrow and the text "BACK", and a blue button with a pencil icon and the text "EDIT".

The screenshot shows a form for the Engine entity. It has a title bar with the text "Engine". Below the title bar, there are two input fields: "ID" with the value "1" and "Engine Number" with the value "KK1234". At the bottom of the form, there are two buttons: a purple button with a left arrow and the text "BACK", and a blue button with a pencil icon and the text "EDIT".

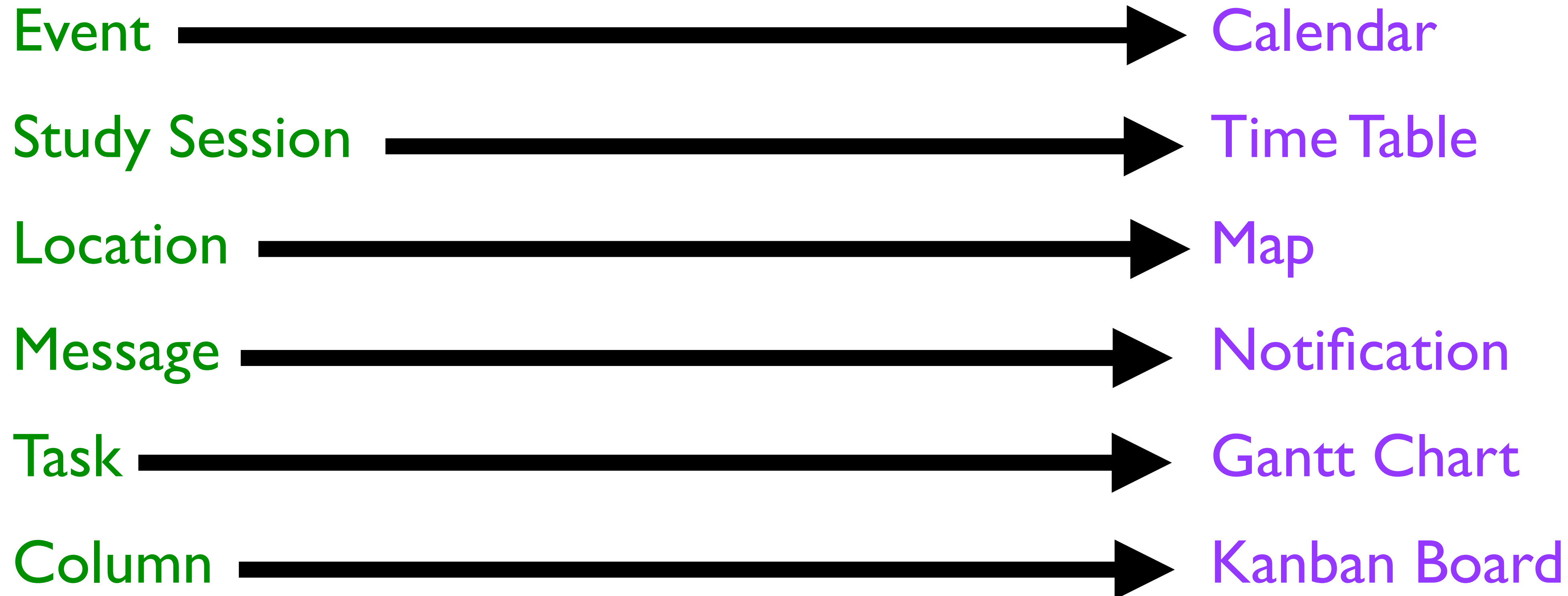
n.b. no spaces in JDL- use *camelCase* labels to get *Title Case*

Issue: How can an Engine ‘know’ which Car it belongs to?

Full Stack Entities vs. Features & Views

Entities :

Views or Features:



who *owns* each of the views?

JHipster: JDL commands

- `jhipster`: generate a new JHipster application. Answer questions about type of application, auth method, database, etc.
- `./mvnw`: start the JHipster application
- `jhipster jdl myapp.jdl`: generate all the entities and relationships specified in `myapp.jdl`
- `jhipster entityName`: Generates a single new entity along with associated files: entity classes, database migration scripts, REST API endpoints, and Angular/React components.

primitive modifiers

```
entity Profile {  
    email String unique required minlength(5) maxlength(25),
```

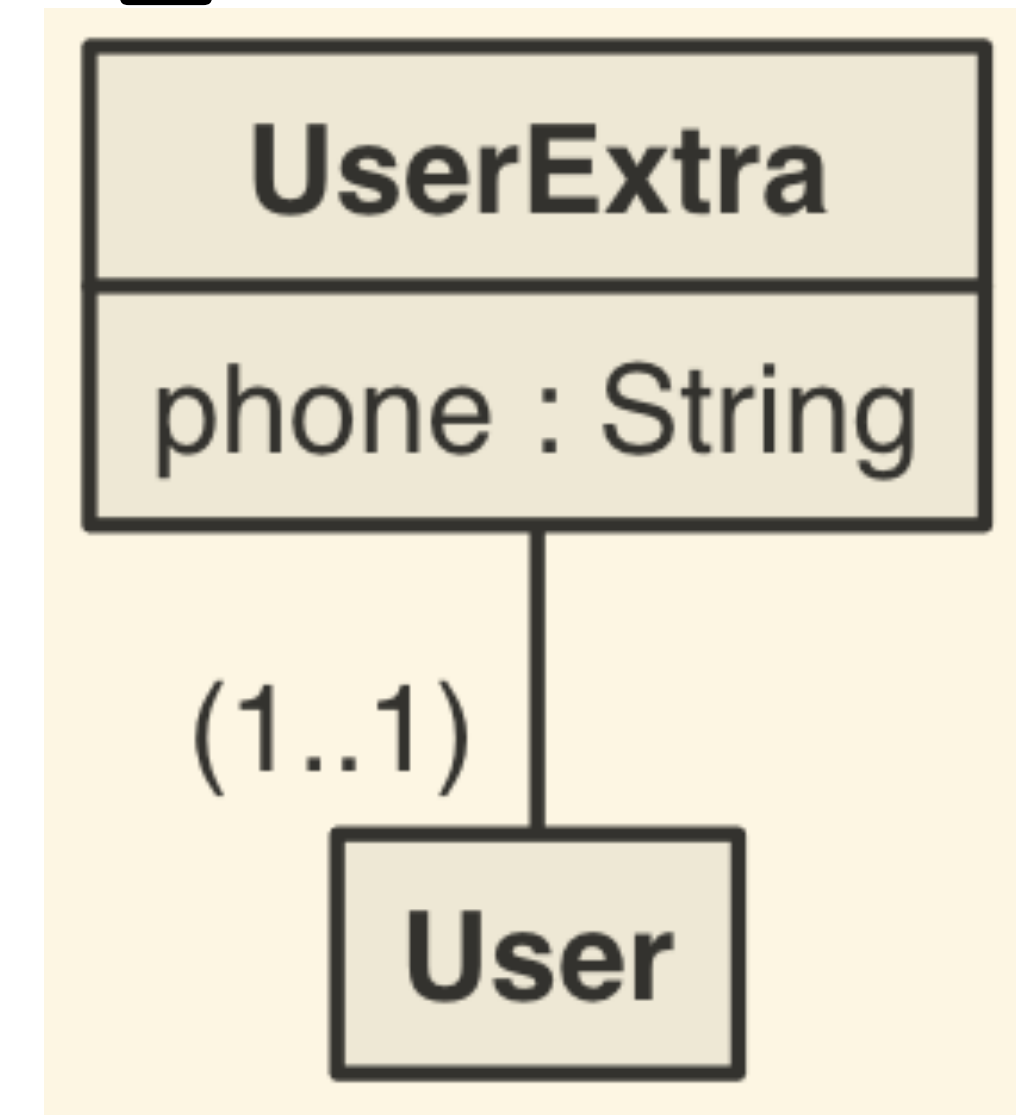
```
entity Event{  
    spaces Integer required min(0) max(150),
```

After each type declaration can have:

- `unique` values must be unique across all instances of the entity
- `required` value is mandatory and cannot be null/empty
- `minlength(n) & maxlength(m)`: length constraints for **string**
- `min(x) & max(y)`: value constraints for Integer

User builtInEntity for login

```
1 entity UserExtra {  
2   phone String  
3 }  
4  
5 relationship OneToOne {  
6   UserExtra{internalUser(login)} to User with builtInEntity  
7 }
```



builtInEntity User cannot be modified:

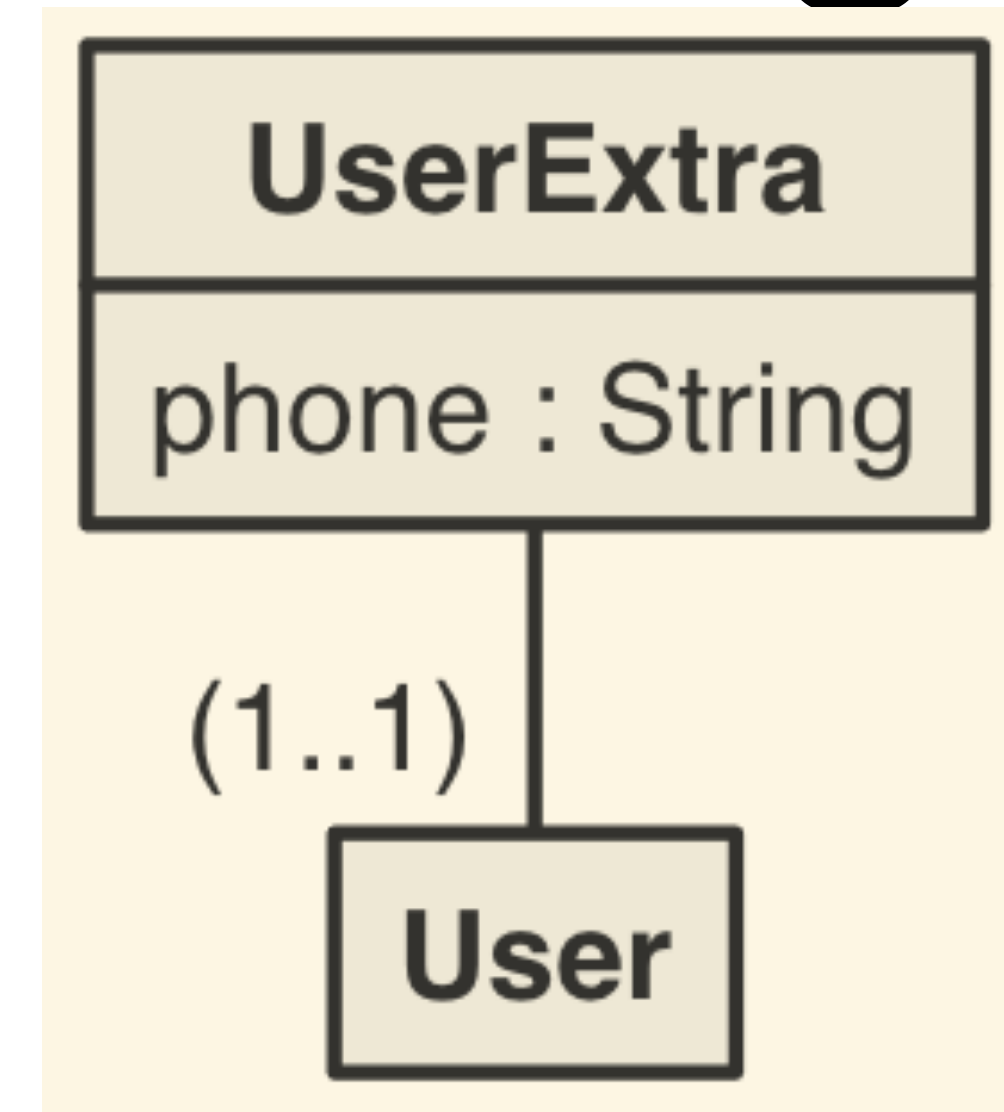
- Fixed Fields: login, first+last name, profile pic, email, password, authorities

To extend with custom fields:

- Add one-to-one relationship to another Entity (e.g. UserExtra)
- (Must use keyword "to User builtInEntity")
- Follow guide to auto create/link UserExtra when a User is created

User builtInEntity for login

```
1 entity UserExtra {  
2   phone String  
3 }  
4  
5 relationship OneToOne {  
6   UserExtra{internalUser(login)} to User with builtInEntity  
7 }
```



builtInEntity User cannot be modified:

- Fixed Fields: login, first+last name, profile pic, email, password, authorities

To extend with custom fields:

- Add one-to-one relationship to another Entity (e.g. UserExtra)
- (Must use keyword "to User builtInEntity")
- Follow guide to auto create/link UserExtra when a User is created

paginate Entity with ...

```
9 // Pagination
10 paginate UserExtra with no
11 paginate UserExtra with pagination
12 paginate UserExtra with infinite-scroll
```

For larger lists of entities, return manageable pages. Analogy: book vs receipt
Instead of returning 10,000 records at once, the API might return 20 at a time.

Performance: Faster page loads and less data transfer.

User Experience: Easier for users to browse with "Next/Previous" buttons.

- no: All records are fetched in one request- risky for large datasets (**default**)
- infinite-scroll: next page loads automatically on scroll like social media feeds

service Entity with serviceClass

```
15 // Service options  
16 service all with serviceClass
```

Create a Service Layer for the named entity (or `all` Entities)

Service contains the business logic or the "rules" of your app

Goes between the Controller (REST requests) and the Repository (database)

Clean Code: Keeps controllers thin and focused on HTTP tasks.

- `no`: No separate service layer, logic directly inside the Controller (**Default**)
- `serviceImpl`: Creates a Service Interface for more advanced OO.

dto all with mapstruct

```
18 // DTO options
19 dto all with mapstruct
```

Create a DTO for the named entity (or `all` Entities)

Data Transfer Objects (DTOs): custom objects to send data. It is **not** the database entity.

mapstruct is a Java library to automatically map DTOs to Entities.

Security: hide sensitive fields that are in your Entity but should never be sent to the client.

Control: shape the data exactly how the front-end needs it, combining data from multiple entities.

filter all

```
18 // DTO options
19 dto all with mapstruct
```

custom backend service for performing filters in the REST API to call the `/api/entity` GET endpoint with the parameters e.g.

`/api/entity?xyz.equals=someValue`

`/api/entity?xyz.in=someValue,otherValue`

For more details, see: <https://www.jhipster.tech/entities-filtering/>

JDL generation is one-shot by default

- Rerunning JDL generation: can **overwrite custom code changes**
- **If the JDL changes:**
 - Simple* changes to entities:
 - edit the JDL (*e.g. add a new entity, field or relationship)
 - Run: `jhipster jdl`
 - Interactively: either overwrite all files or keep some modified files.
 - `help` and `diff` commands are very useful to find out what changes will be made

Regenerating specific entities: jhipster entity

- Alternative: update only the entity with changes by running jhipster entity
- **If an Entity changes:**
 - Use the jhipster entity EntityName command and then review the prompts about changes.
 - *"Do you want to update the entity? This will replace the existing files for this entity, all your custom code will be overwritten"*

Regenerating the whole app: git cherry pick/checkout

- **Big changes to JDL:**
- Sometimes JDL changes are too complex to regenerate:
 - The previous app should first go into a git repo
 - Then a new template app can be generated from updated JDL.
 - Important git history can be copied over using git cherry-pick (specific commits) or git checkout (specific files)

Getting started with JHipster Development

JHipster: dev commands

- `jhipster` : generate a new JHipster application. Answer questions about type of application, authentication method, database, etc.
- `jhipster ci-cd` : generate a ci and cd pipelines JHipster answer questions about type of ci pipeline needed
- `jhipster jdl myapp.jdl` : generate all the entities and relationships specified in `myapp.jdl`
- `jhipster entity` : Generates a single new entity along with associated files: entity classes, database migration scripts, REST API endpoints, and Angular/React components.

JHipster: run commands

- `./mvnw` : Runs the application in development mode (port 8080)
- `npm install` : installs node needed packages
- `npm start` : starts front end only (port 9000)

JHipster: important files

- `.github/workflows/main.yml`: GitHub pipeline
- `.gitlab-ci.yml`: gitlab ci pipeline
- `pom.xml`: MVN configuration file for managing dependencies and building the Java application.
- `src/main/docker/dev.yml` : the docker compose script to start the app and postgres on a the server - development mode
- `src/main/docker/prd.yml` : the docker compose script to start the app and postgres on a the server - development mode

Agile Web Development: task

Regenerating from JDL

- JDL generation is generally one way
- Add entity or field: use `jhipster jdl` or `jhipster entity` command to modify entities e.g. to add an attribute
- Rename or remove entity, field or relationship: re-start from template:
 - re apply previous commits or files changed
 - Set `jhipster` command not to overwrite specific files

Unexpected Keywords

- Home, Authority, Profile, Footer are keywords in JDL